FUN GC ／ UNITY講座シリーズ

【Unity基礎】

オブジェクト指向をC#で学ぼう

第1回　クラスとインスタンス



2026年10月19日　講師：嘉屋悠月

TODAY ・ 今日のゴール

3つできるようになろう



設計図が書ける

class で、敵の設計図を書ける



実物を作れる

new や Instantiate で、設計図から実物を生み出せる



増やせる

スペースキーを押すたびに敵が増えるしくみを作れる

POINT　用語は後から。まず動かして体験してから、名前を付けよう。

TODAY ・ 90分

今日の流れ

00	前回の演習の続き	8分
01	クラスは「設計図」	16分
02	new でインスタンスを作る	16分
03	Unityの中のクラス	15分
04	演習：スペースキーで敵を出す	30分
05	まとめ・次回予告	5分

EXERCISE

演習

目安時間

8 分

前回の続き

Mover を完成させよう

ゴール　矢印キーでキューブが動き、
その速さをインスペクターから変えられる

1. Mover.cs を開く（リポジトリの 05_mover）
2. public float speed = 5f; を書く
3. Update() で、if と Input.GetKey で矢印キーを調べる
4. **押されていたら x や z に 1 か -1 を入れて動かす**

早く終わった人へ

スペースキーでジャンプさせる ／ 画面の外に出たら
Console に表示する

今日のコードは clone と pull で読み込む

1. GitHub Desktop で「File → Clone repository」。yugetu12/fungc-unity-course を選ぶ（初回だけ。Unity プロジェクトとは別の場所に）
2. **左上の「Current Branch」で unity-base-1 に切り替える**（2回目からは「Fetch origin → Pull」）
3. clone したフォルダの scripts にある .cs を、Unity の Project ウィンドウにドラッグ＆ドロップ
4. MonoBehaviour は Hierarchy にドラッグ。再生して動きを確かめる

⚠ Spawner は1つだけ

03_prefab と 04_exercise の Spawner.cs は、どちらか1つだけ入れる。両方入れると同じ名前でエラーになる。

GitHub Desktop が使えない人は、GitHub の「Code → Download ZIP」でもOK。

RECAP ・ 前回のふりかえり

前回書いたスクリプト、覚えてる？

Mover.cs（第0回）

```
public class Mover : MonoBehaviour
{
    public float speed = 5f;

    void Update()
    { /* 矢印キーで動く */ }
}
```

この class って、何だろう？

前回は「おまじない」として
書いていたこの1語が、
今日の主役です。

SECTION

01

クラスは「設計図」



モノの作り方を書いたものがクラス。
まずは Unity の機能を使わない、ふつうの C# で見てみよう

クラスは「設計図」、インスタンスは「実物」

クラス（class）

どんなデータと動きを持つかを書いた設計図。
設計図だけでは、まだゲームに出てこない。

インスタンス

設計図から作った実物。
ゲームの中で実際に動くのはこっち。

たとえるなら

たい焼きの「型」がクラス、
焼き上がった1匹1匹がインスタンス。



クラスの中身：フィールドとメソッド

Enemy.cs

```
public class Enemy
{
    public string name;
    public int hp;

    public void Attack()
    {
        Debug.Log(name + "の攻撃！");
    }
}
```

① フィールド ＝ データ

name や hp のように、クラスの中に
書いた変数。実物ごとに値を持つ。

② メソッド ＝ 動き

前回の「関数」と同じもの。
クラスの中の関数をメソッドと呼ぶ。

③ MonoBehaviour なし

「: MonoBehaviour」を書かない、
ふつうの C# のクラス。

SECTION

02

new で
インスタンスを作る



設計図から実物を作る合言葉が new。
作った実物には、1つずつ別の値を入れられる

new で作って、「.」で値を入れる

GameManager.cs ／ class の中

```
void Start()
{
    Enemy slime = new Enemy();
    slime.name = "スライム";
    slime.hp = 10;

    Enemy goblin = new Enemy();
    goblin.name = "ゴブリン";
    goblin.hp = 30;

    slime.Attack();
    goblin.Attack();
}
```

① new クラス名()

設計図から実物を1つ作る。
Enemy 型の変数に入れて使う。

② 変数名.フィールド

「slime の hp」のように、
どの実物かを「.」でつなぐ。

▷ Console ／ 実行結果

```
スライムの攻撃！
ゴブリンの攻撃！
```

形は同じ、中の値は1体ずつ別々

CLASS ／ 設計図

Enemy

```
name = ?
hp = ?
Attack()
```

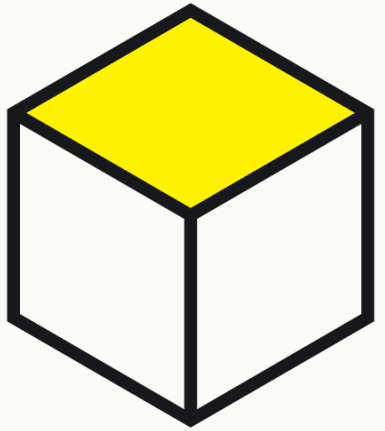
new

slime	name = "スライム" hp = 10
goblin	name = "ゴブリン" hp = 30
boss	name = "ドラゴン" hp = 500

…いくつでも

POINT ここだけは覚える

クラスは1つ。
インスタンスはいくつでも。



敵が100体いても、書くクラスは Enemy の1つだけ。

よくあるつまずき：設計図に値を入れる

⚠ NG　エラーになる

```
Enemy.hp = 10;
```

Enemy はクラス（設計図）。
どの敵の hp なのか決まらない。

✓ OK　正しく動く

```
slime.hp = 10;
```

new で作ったインスタンス
slime の hp を変える。

エラー CS0120 が出たら、「クラス名.」と書いていないか確認しよう。

SECTION

03

Unityの中のクラス

じつは第0回から、ずっとクラスとインスタンスを
使っていた。その正体を見てみよう



アタッチ ＝ インスタンスを作ること

Mover.cs ＝ クラス

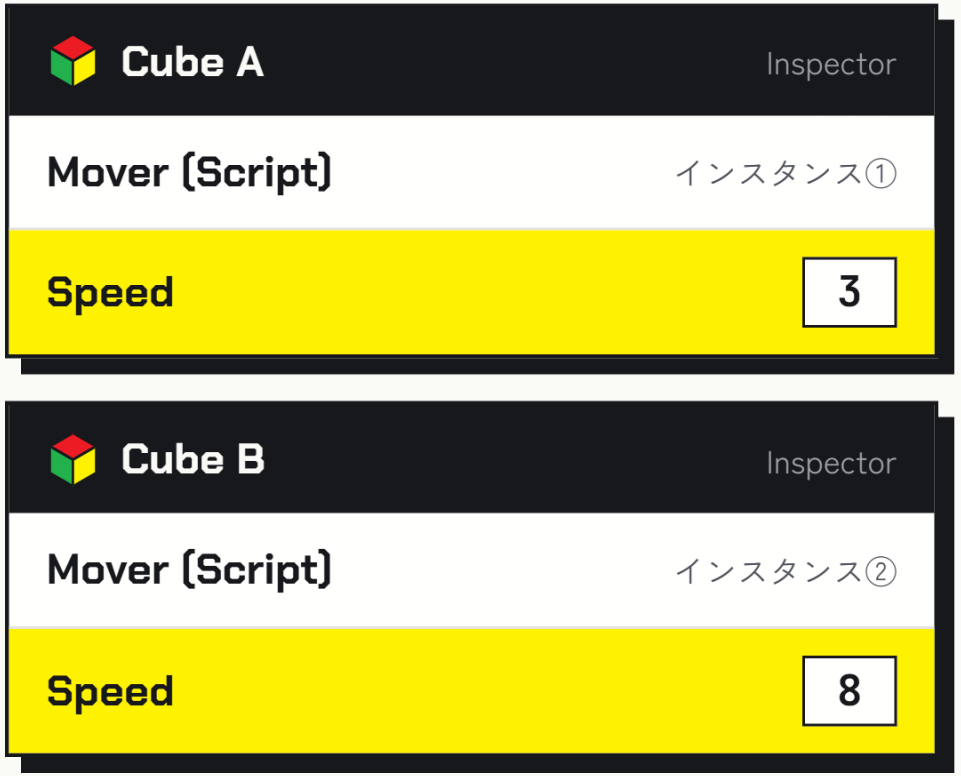
前回書いたスクリプトは、
「キューブの動かし方」の設計図。

アタッチ ＝ new

GameObject に付けるたびに、
Unity が実物を1つ作ってくれる。

だから値が別々

キューブAとBで speed を変えても、
お互いに影響しない。



注意：MonoBehaviour は new しない

⚠ NG　正しく動かない

```
Mover m = new Mover();
```

Unity の部品は new では作れない。
Console に警告が出る。

✓ OK　アタッチする

```
gameObject.AddComponent<Mover>();
```

エディタでアタッチするのと同じ。
Unity が実物を作ってくれる。

ふつうのクラスは new、MonoBehaviour はアタッチ。この使い分けを覚えよう。

Prefab と Instantiate：Unity 版の new

Spawner.cs

```
public class Spawner : MonoBehaviour
{
    public GameObject enemyPrefab;

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space))
        {
            Instantiate(enemyPrefab);
        }
    }
}
```

① Prefab ＝ 設計図

GameObject をまるごと保存したもの。
Project ウィンドウへドラッグで作れる。

② Instantiate ＝ new

Prefab から、シーンに実物を
1つ作る。何回でも呼べる。

③ GetKeyDown

押した瞬間だけ true。
押しっぱなしでも1回だけ。

EXERCISE

演習

目安時間

30分

スペースキーで敵を出そう

ゴール　スペースを押すたびにキューブが増え、
矢印キーで動かすと1体ずつ速さが違う

1. 前回の Mover 付きキューブを Prefab にする
2. 04_exercise の Spawner.cs を読み込み、空の GameObject にアタッチ
3. enemyPrefab 欄に Prefab を入れ、スペースで確認
4. **TODO 4 を自分で書いて、speed をランダムにする**

早く終わった人へ

出した数を Console に表示する ／ Enemy クラスに
Heal() メソッドを足して、slime だけ回復させる

解答例：Spawner.cs

Spawner.cs

```
public class Spawner : MonoBehaviour
{
    public GameObject enemyPrefab;

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space))
        {
            GameObject obj = Instantiate(enemyPrefab);
            Mover m = obj.GetComponent<Mover>();
            m.speed = Random.Range(1f, 8f);
        }
    }
}
```

① 作った実物を受け取る

Instantiate は、今作ったインスタンスを返してくれる。

② GetComponent<Mover>()

そのキューブに付いている Mover のインスタンスを取り出す。

③ 1体ずつ値を入れる

m.speed を変えても、ほかのキューブの speed は変わらない。

05 ・ まとめ

今日できるようになったこと



クラス ＝ 設計図

フィールド（データ）とメソッド（動き）をひとまとめにしたもの



インスタンス ＝ 実物

new で作る。いくつでも作れて、値は1つずつ別々



Unity では

スクリプトはクラス。アタッチと Instantiate がインスタンスを作る

NEXT

第2回　カプセル化

今日は hp を自由に書き換えられた。次回は、外から壊されないように守る書き方を学びます。